Paper review · Archeologist

# Where FAST came from, and what it led to

FAST: Efficient Action Tokenization for Vision-Language-Action Models. Pertsch, Stachowicz, Ichter, Driess, Nair, Vuong, Mees, Finn and Levine. Physical Intelligence, UC Berkeley and Stanford.

---

arXiv:2501.09747 · January 2025

Pavel Bushuyeu · October 2026

bushuyeu.com/review-fast-lineage

# FAST in one slide

## The problem

RT-2 and OpenVLA turn each action dimension at each time step into one of 256 bin tokens. At high control rates neighbouring tokens are almost the same, so the model learns to copy the last token.

## The fix

Compress each one-second action chunk before tokenizing: a discrete cosine transform per dimension, scale and round as JPEG does, then byte-pair encoding.

π0-FAST matches diffusion π0 with **5× fewer GPU hours** of training, but needs about **750 ms** per action chunk at inference against about 100 ms (NVIDIA 4090).

| Dataset | Rate | Binning | FAST | Ratio |
|---|---|---|---|---|
| **BridgeV2** | 5 Hz | 35 | 20 | 1.75× |
| **DROID** | 15 Hz | 105 | 29 | 3.6× |
| **Table bussing** | 20 Hz | 140 | 28 | 5.0× |
| **Shirt folding** | 50 Hz | 700 | 53 | 13.2× |

Tokens per one-second chunk at similar reconstruction error. FAST, Table I.

# The FAST tokenizer

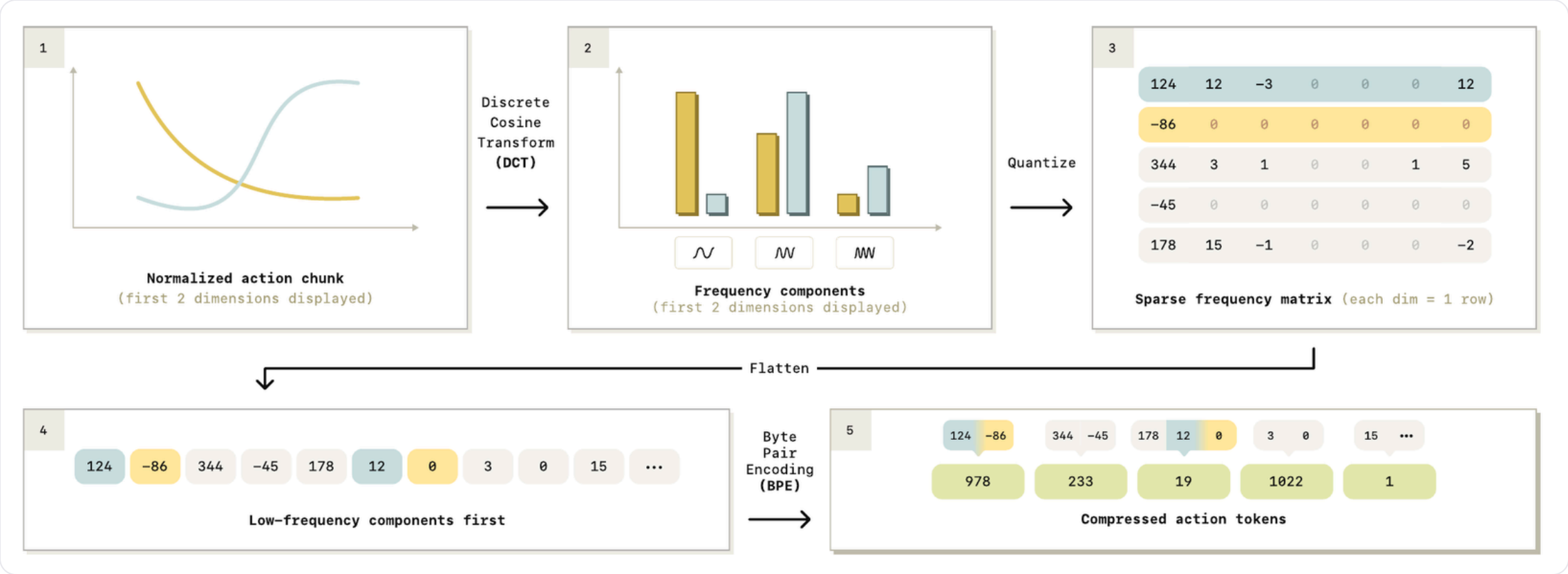
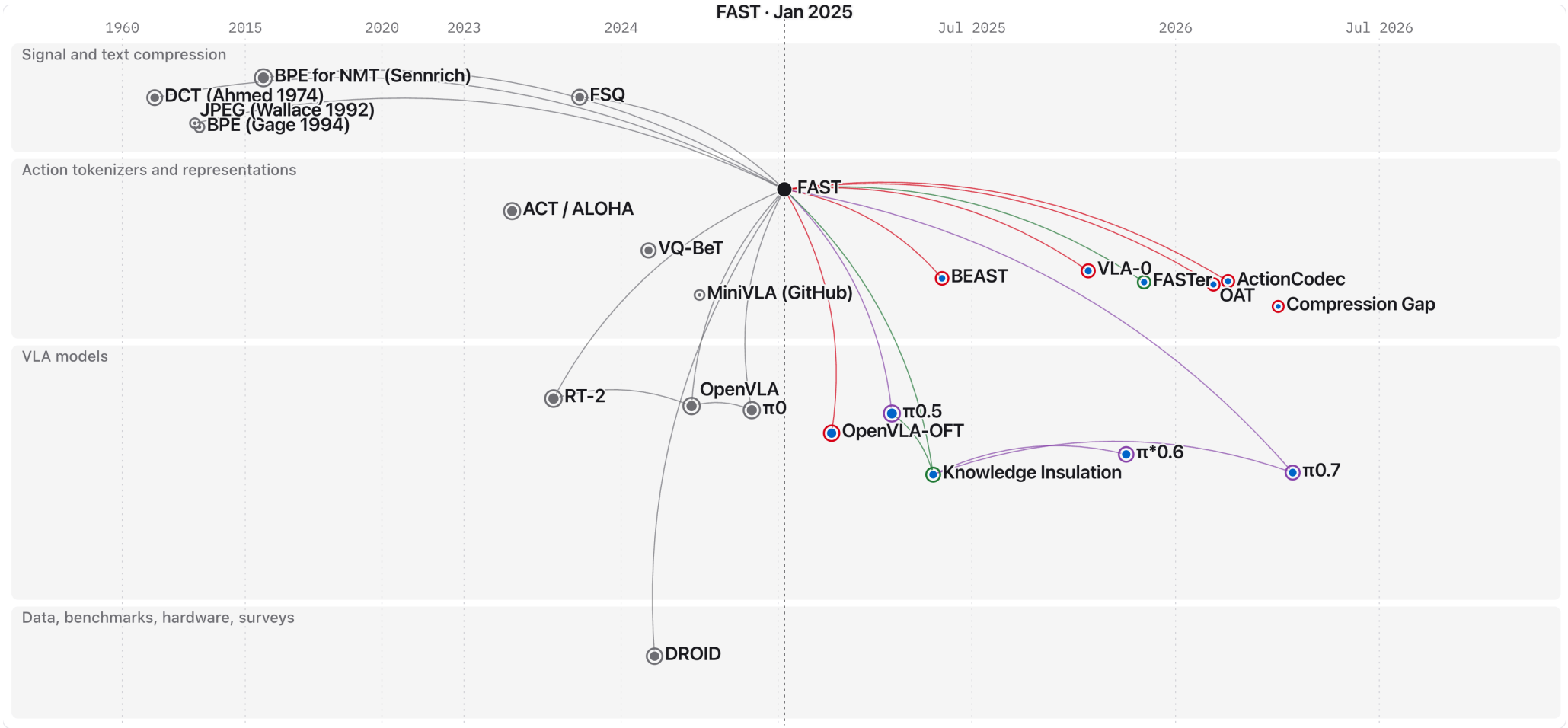



Fig. 4 of FAST: DCT per action dimension, quantization to a sparse matrix, flattening with the low frequencies first, then byte-pair encoding. Pertsch et al., arXiv:2501.09747, CC BY 4.0.

# Citation timeline: the papers in this talk




FAST's main sources (left) and the later papers that extend, validate or challenge it (right). Full timeline in the appendix.

# Prior works

| | |
|---|---|
| 1974–2016 | **DCT, JPEG and byte-pair encoding**<br>The compression idea, from outside robotics: a frequency transform, rounding, then BPE. |
| 2017–2024 | **VQ-VAE, FSQ, VQ-BeT, MiniVLA**<br>The road not taken: FAST argues learned quantizers struggle with fine control; its FSQ ablation underperforms on high-frequency tasks. |
| Apr 2023 | **ACT and ALOHA**<br>Action chunking: predict several future actions together. |
| Jul 2023 | **RT-2**<br>The first VLA: one bin token per action dimension per time step. |
| Mar 2024 | **DROID**<br>Binning-based VLAs trained poorly on it; FAST calls its DROID policy the first to work zero-shot in unseen scenes. |
| Jun 2024 | **OpenVLA**<br>An open 7B VLA that kept binning; “OpenVLA-style” binning is FAST’s main baseline. |
| Oct 2024 | **π0**<br>The same lab’s earlier model: FAST’s base and its point of comparison. |

# OpenVLA and RT-2, then π0

## Binning, kept and replaced

- FAST keeps their recipe: a VLM fine-tuned to write action tokens, trained with next-token prediction.
- Its main baseline is “OpenVLA-style” per-dimension binning (Fig. 2), mostly on the same π0 backbone.
- Karl Pertsch, FAST’s first author, co-wrote RT-2, OpenVLA, Octo and DROID.

## π0: same lab, the base model

π0 is PaliGemma-3B with a flow-matching action expert. π0-FAST keeps the backbone, drops the expert and writes FAST tokens into the VLM’s vocabulary.

The paper compares the two decoders on one model: about 5× less training compute for π0-FAST, about 750 ms against 100 ms per chunk at inference (NVIDIA 4090).

After FAST

# What came after

| | | |
|---|---|---|
| Apr 2025 | **π0.5** • Validates | Robot actions as FAST tokens in pre-training; a flow-matching expert added in post-training. 8 of FAST's 9 authors. |
| May 2025 | **Knowledge Insulation** • Extends | One stage instead of two: FAST tokens and a flow-matching expert trained together, with a stop-gradient. |
| Jun 2025 | **Real-time chunking** • Related | Removes the pause between chunks for diffusion and flow policies, at inference only. |
| Nov 2025 | **π*0.6** • Validates | Keeps FAST tokens in training through Knowledge Insulation. |
| Dec 2025 | **FASTer** • Extends | A learned VQ tokenizer that treats an action chunk as an image. |
| Apr 2026 | **π0.7** • Validates | Its VLM backbone "is supervised with FAST tokens", through Knowledge Insulation. |

Extends: builds on FAST's method. Validates: a later system adopts FAST tokens, not an independent test. Related: same problem, not built on FAST.

# Knowledge Insulation does FAST's future work

> "...testing the combination of compression-based action encodings with non-autoregressive decoding approaches like diffusion..."
>
> FAST, discussion of future work

- **Knowledge Insulation** (May 2025; Driess, Pertsch, Levine and others) does this in one stage: the VLM learns from FAST tokens, a flow-matching expert produces the actions, and a stop-gradient keeps the expert from damaging the VLM.
- Train fast like π0-FAST, run fast like π0. π0.5 had used FAST tokens in pre-training and added the expert afterwards; π*0.6 and π0.7 keep the one-stage recipe.

# FAST against its challengers

| Paper | Objection to FAST | Uses instead | FAST against it, in that paper's setup |
|---|---|---|---|
| **OpenVLA-OFT**<br>Feb 2025 | ~750 ms between chunks is too slow for high-frequency bimanual robots. | Continuous actions, L1 loss, parallel decoding. | LIBERO, fine-tuned: π0-FAST **85.5%**, π0 94.2%, OFT **97.1%**. |
| **BEAST**<br>Jun 2025 | Variable token counts "complicate" parallel decoding. | A fixed number of B-spline tokens per chunk. | Same Florence-2, no robot pre-training: BEAST-AR ties FAST on LIBERO-Long; BEAST-PD does best. |
| **VLA-0**<br>Oct 2025 | Custom tokenizers "often require intricate training pipelines". | Actions as plain-text numbers. | LIBERO: VLA-0 **94.7%** without robot pre-training; OFT 91.9% without, 97.1% with. |

Each paper's own setup: compare within a row. LIBERO = average over four suites.

Comparison 2 of 2 · the tokenizer itself

# FAST against its challengers

| Paper | Objection to FAST | Uses instead | FAST against it, in that paper's setup |
|---|---|---|---|
| **ActionCodec**<br>Feb 2026 | "Fixed geometric priors"; decoded length must match, so outputs are padded or cut. | A learned tokenizer. | Same SmolVLM2-2.2B, no robot pre-training: FAST **90.6%**, best of five; ActionCodec **95.5%**. |
| **OAT**<br>Feb 2026 | Invalid sequences make the policy "unstable, overly aggressive". | Learned ordered tokens; any prefix decodes. | Small policies from scratch, LIBERO-10 only: FAST **23.0%**, OAT-8 **56.3%**. |
| **Compression Gap**<br>Apr 2026 | A fixed-capacity code can cap encoder gains. | An analysis. | Not tested on FAST. With OAT, encoder upgrades add +3.6 and +10.4 points. |

Each paper's own setup: compare within a row. ActionCodec: four-suite LIBERO average; OAT: LIBERO-10 only.

# What the comparison shows

- **As a tokenizer, FAST is competitive.** It beats binning and VQ tokenizers but trails ActionCodec (90.6% vs 95.5%) and BEAST's parallel variant.
- **Decoding is the common objection.** Sequences that do not decode (OAT, ActionCodec), the delay per chunk (OpenVLA-OFT), variable length (BEAST).
- **Training setups differ.** The 86.0% vs 71.8% pre-training gap comes from separate reports; FAST reached 90.6% with no robot pre-training (ActionCodec).
- **Its own lab answered the speed objection.** π0.5, Knowledge Insulation, π*0.6 and π0.7 train on FAST tokens and act with flow matching.

# OpenVLA-OFT

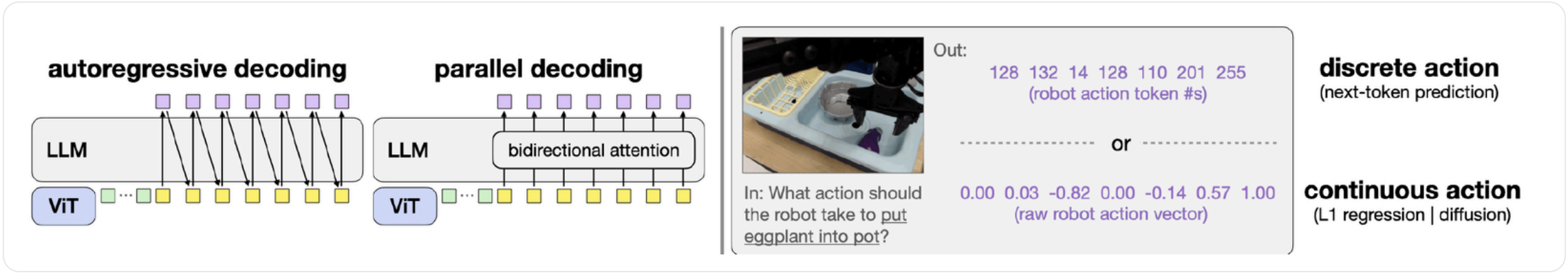



Fig. 2 of OpenVLA-OFT: autoregressive against parallel decoding (left), discrete action tokens against continuous actions (right). Kim, Finn and Liang, arXiv:2502.19645, CC BY 4.0.

# BEAST: the tokenizer

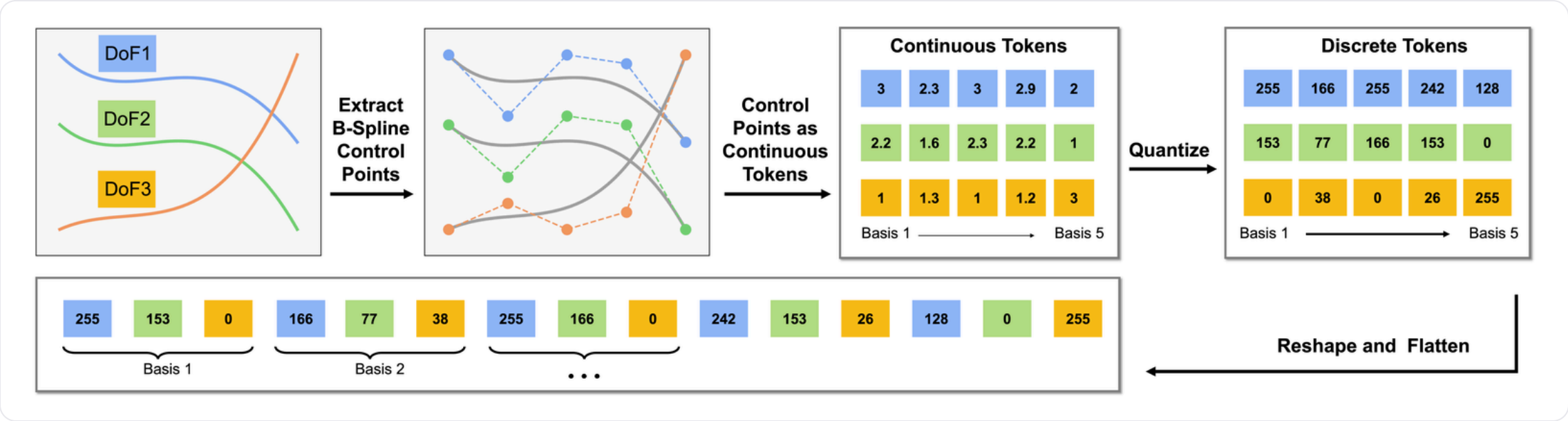



Fig. 2 of BEAST: B-spline control points of each action dimension become continuous tokens, then discrete ones, flattened into a sequence of fixed length. Zhou et al., arXiv:2506.06072, CC BY-NC-SA 4.0.

# BEAST-F: the model

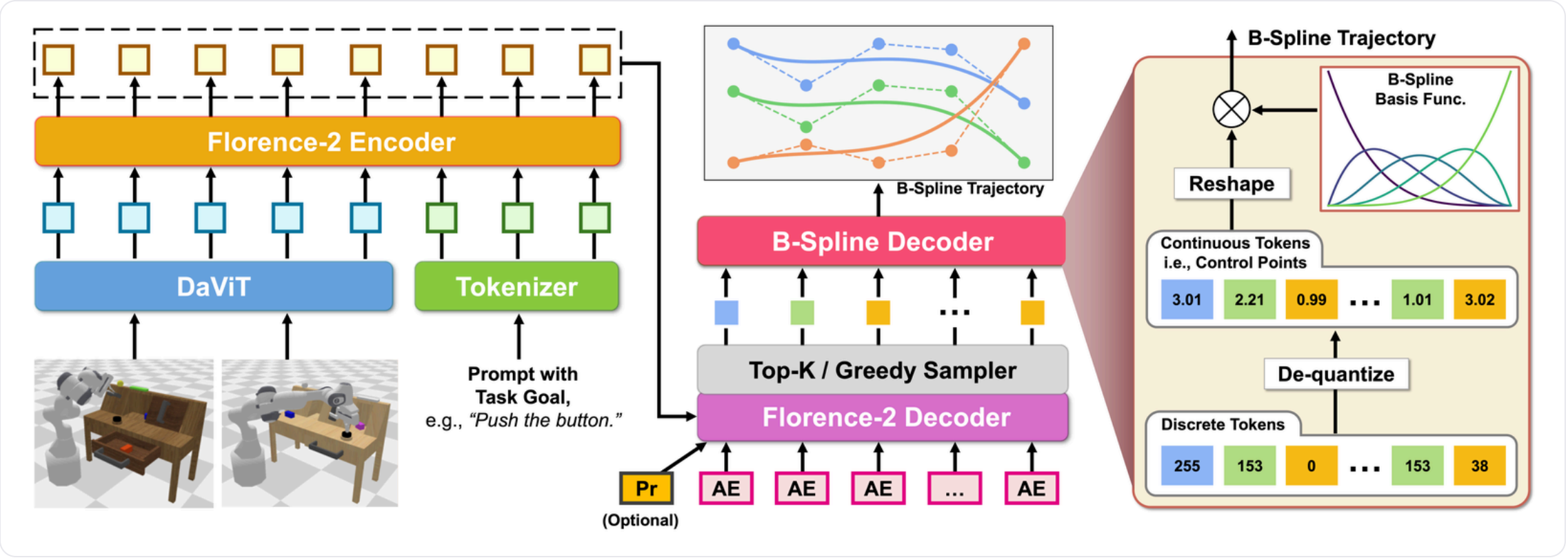



Fig. 3 of BEAST: BEAST with the Florence-2 VLM; a fixed number of tokens per chunk allows parallel decoding. Zhou et al., arXiv:2506.06072, CC BY-NC-SA 4.0.

# VLA-0

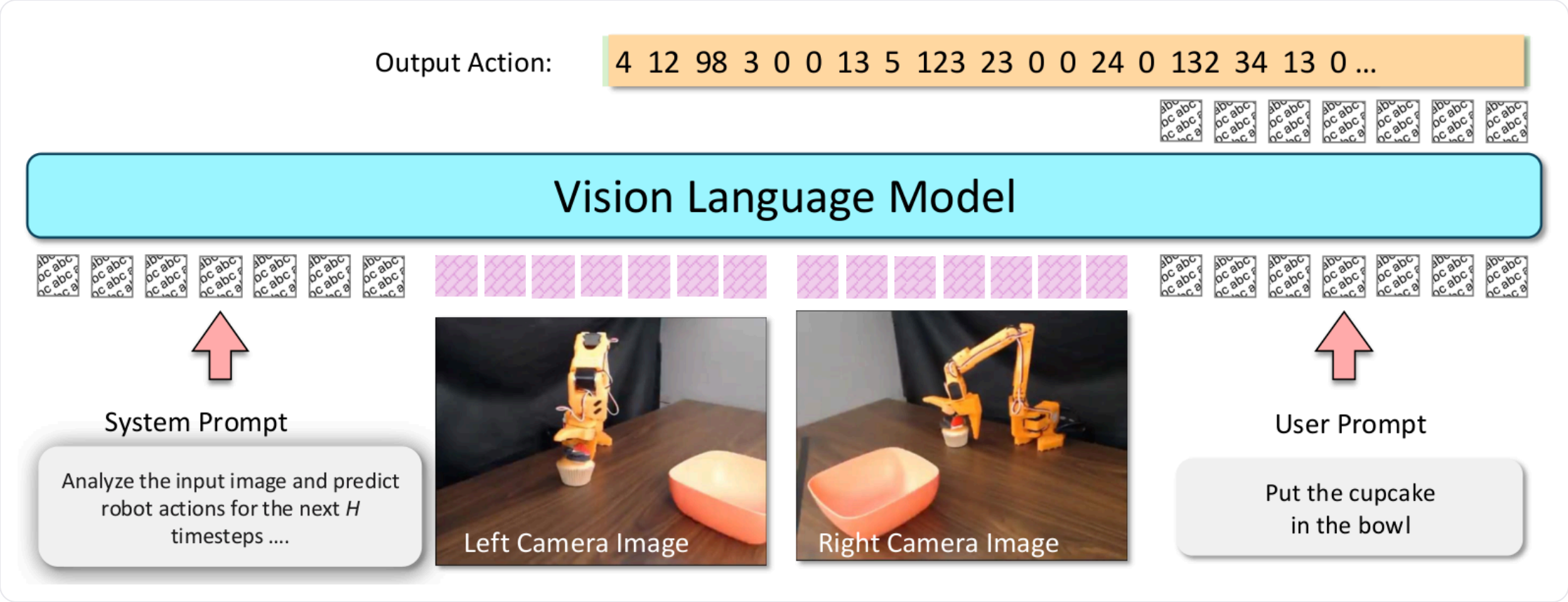



Fig. 3 of VLA-0: the VLM is not changed; it writes actions as space-separated integers. Goyal et al., arXiv:2510.13054, arXiv non-exclusive license; shown for commentary.

# ActionCodec

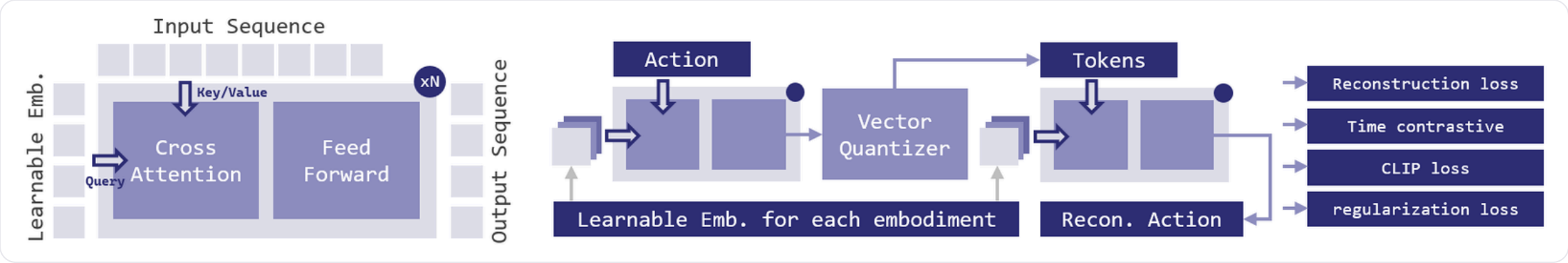



Fig. 2 of ActionCodec: a Perceiver-like encoder and decoder around a vector quantizer, with an embedding per embodiment and four training losses. Dong et al., arXiv:2602.15397, CC BY 4.0.

# OAT

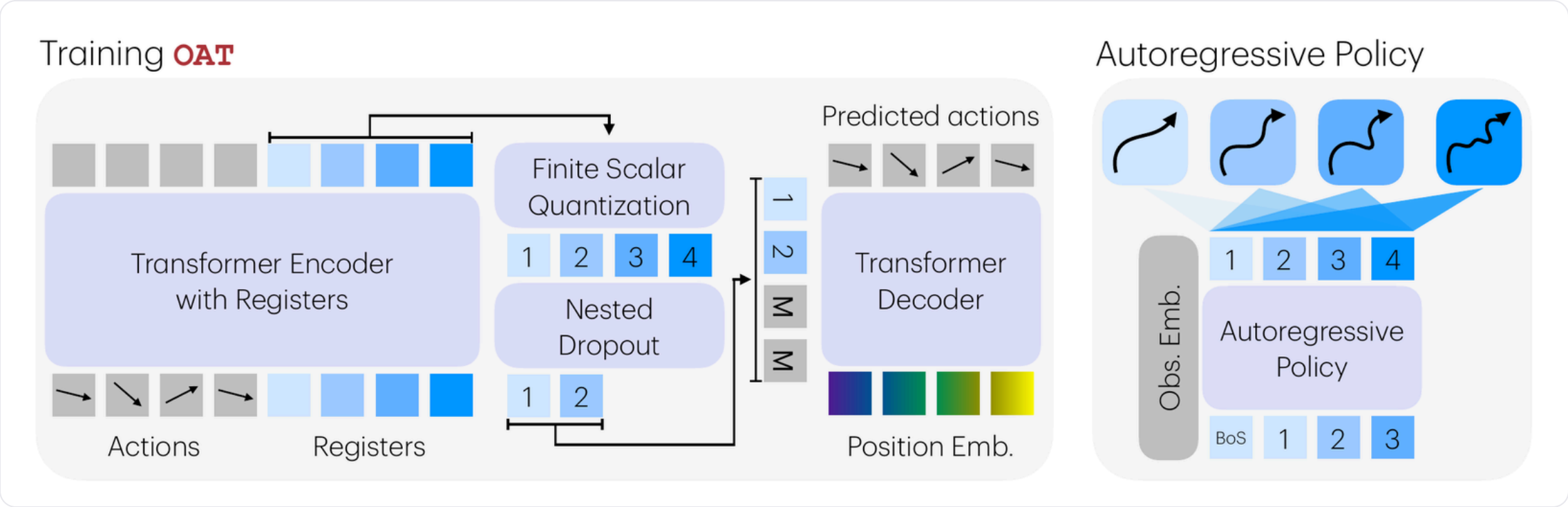



Fig. 3 of OAT: ordered tokens from a transformer encoder with registers, FSQ and nested dropout (left), generated by an autoregressive policy (right). Liu et al., arXiv:2602.04215, CC BY 4.0.

# The Compression Gap

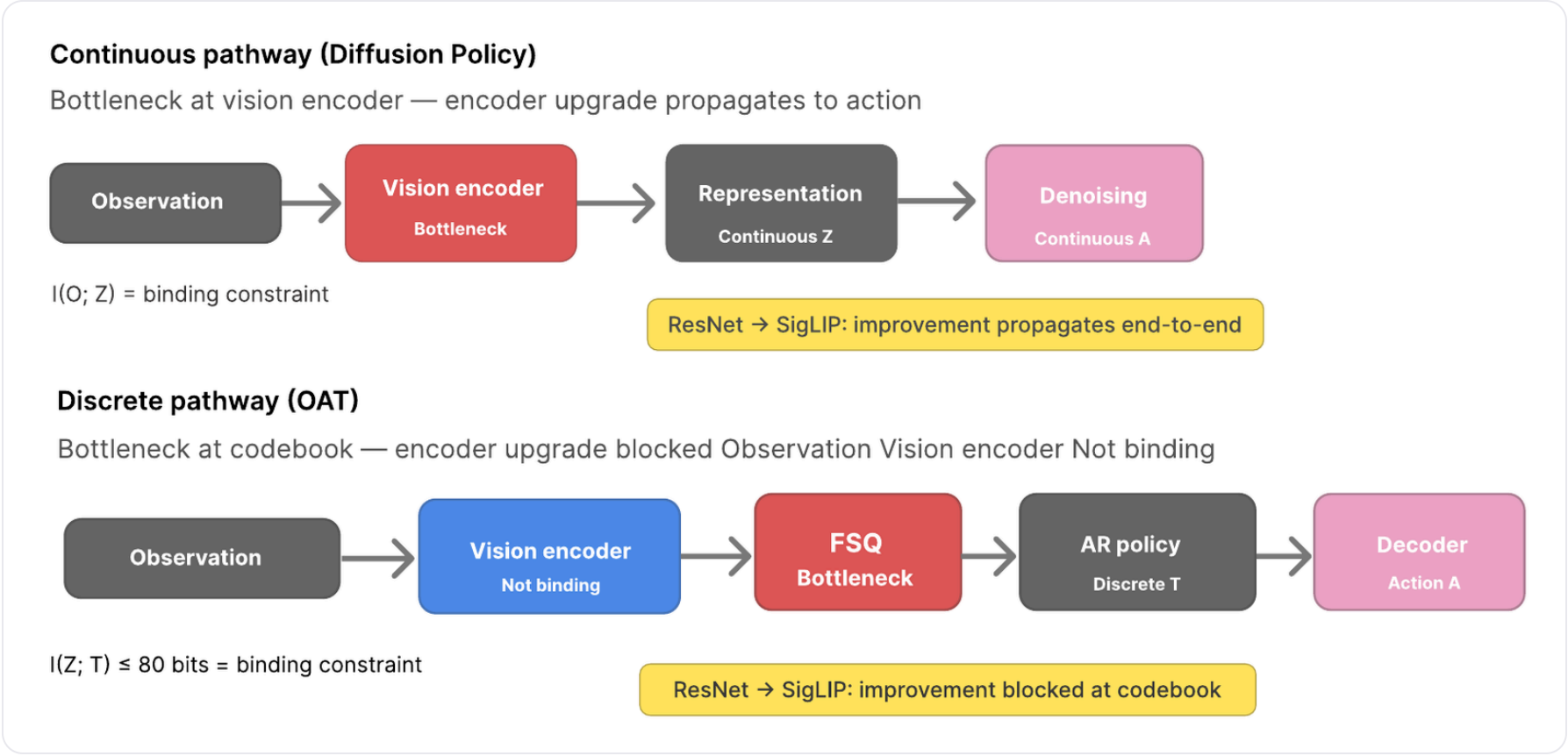
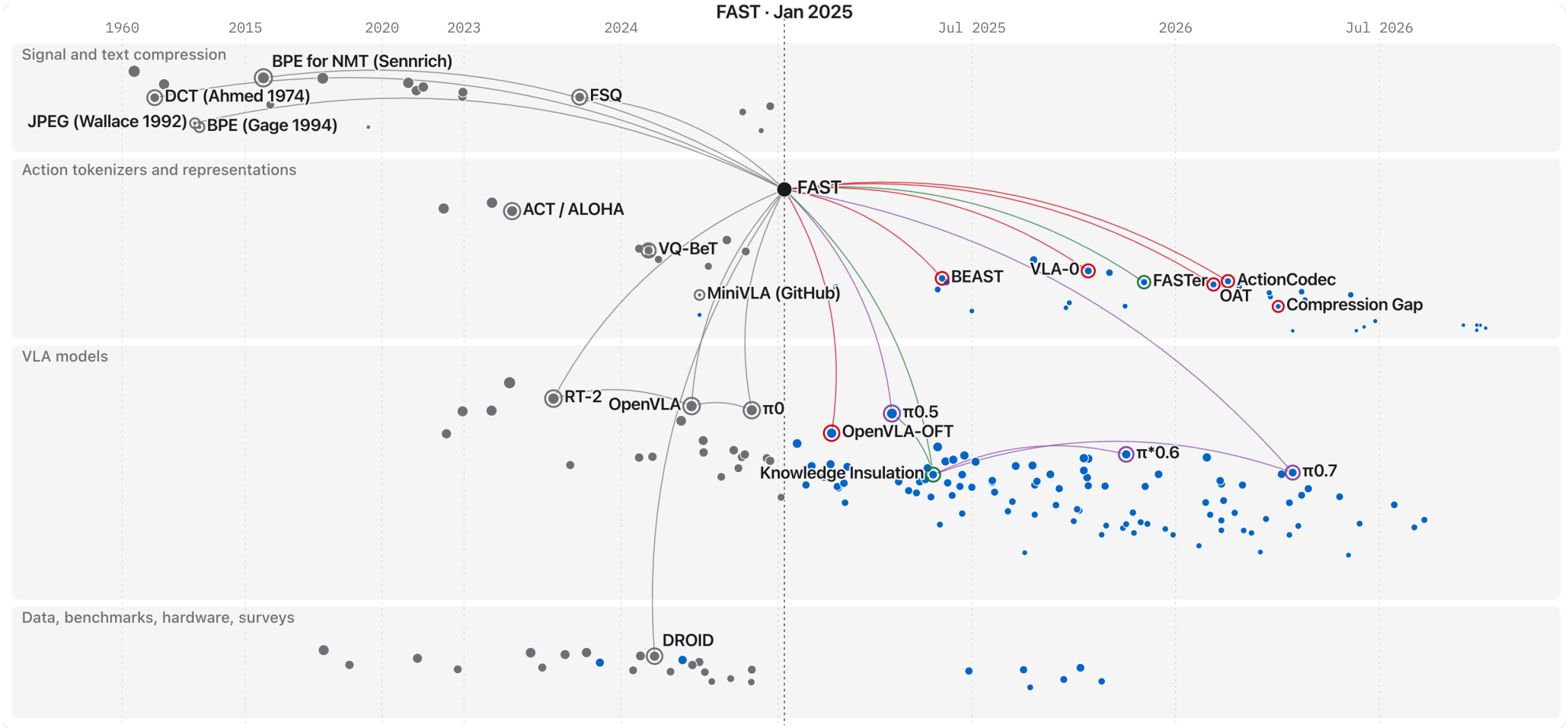



Fig. 1 of The Compression Gap: the author's explanation for smaller encoder gains with discrete tokens; with OAT's codebook (about 80 bits per chunk in the tested setting) the code may be the bottleneck. Shiba, arXiv:2604.03191, CC BY 4.0.

# Full citation timeline




72 references and 140 of 781 citing papers, by first publication date; within each lane, higher means more cited. Semantic Scholar, 19 Oct 2026.

# Sources


1. Pertsch et al. FAST, arXiv:2501.09747, 2025: Table I, Figs. 2 and 4, §VI.
2. Semantic Scholar Graph API, read 19 Oct 2026: citing papers, citing sentences, reference lists.
3. Ahmed et al. Discrete cosine transform, 1974; Wallace. JPEG, 1992.
4. Gage. BPE, 1994; Sennrich et al. arXiv:1508.07909.
5. Mentzer et al. FSQ, 2023; Lee et al. VQ-BeT, 2024.
6. Zhao et al. ACT / ALOHA, 2023.
7. Brohan et al. RT-2, 2023.
8. Khazatsky et al. DROID, 2024.
9. Kim et al. OpenVLA, 2024.
10. Black et al. π0, 2024.
11. Physical Intelligence. π0.5, 2025.
12. Driess et al. Knowledge Insulation, 2025.
13. Black et al. Real-time chunking, 2025.
14. Physical Intelligence. π*0.6, 2025; π0.7, 2026.
15. Liu et al. FASTer, 2025.
16. Kim, Finn and Liang. OpenVLA-OFT, 2025: LIBERO table.
17. Zhou et al. BEAST, 2025: §5.4, Figs. 7 and 8.
18. Goyal et al. VLA-0, 2025: Table I.
19. Dong et al. ActionCodec, 2026: LIBERO table, App. F.1.
20. Liu et al. OAT, 2026: Table I (LIBERO-10).
21. Shiba. The Compression Gap, 2026: Table 1.
22. This review, with the interactive timeline: bushuyeu.com/review-fast-lineage.